TALLER DE CIBERSEGURIDAD

Laboratorio Man-in-the-Middle en Redes SDN

Guía paso a paso ilustrada: cómo montar y presentar el laboratorio de interceptación y defensa de tráfico en una red definida por software.

5 máquinas virtuales | VirtualBox + Ubuntu 22.04 | Un solo comando: bash lab.sh | Duración ≈ 2 horas

---

Repositorio: github.com/juanortizpa/demostracion-narino • Charla para estudiantes de Nariño
Uso exclusivamente educativo, en un entorno virtual y aislado.

¿Qué vamos a mostrar?

LA IDEA

El laboratorio cuenta una historia en cuatro momentos. Primero una comunicación normal; luego cómo un atacante se mete en medio para **ver** y **alterar** los mensajes; y por último, lo más importante, **cómo se detecta y se bloquea** ese ataque.



Figura 1 — La narrativa del laboratorio en cuatro momentos.

El mensaje clave para los estudiantes: la ciberseguridad no es solo teoría. El mismo conocimiento que permite entender un ataque es el que permite **defender** a las personas y organizaciones. El momento 4 es el trabajo real de un profesional en ciberseguridad.

Las 5 máquinas virtuales

VM	Rol	IP(s)	Qué hace
ryu	Controlador SDN	192.168.100.10	El "cerebro" de la red. Decide y, en modo defensa, bloquea.
ovs	Switch OpenFlow	192.168.100.20 / 10.0.0.1	El conmutador que mueve los paquetes.
h1	Víctima A	10.0.0.11	Envía el mensaje.
h2	Víctima B	10.0.0.12	Recibe el mensaje.
h3	Atacante	10.0.0.66	Se mete en medio: ve y altera.

Cómo se conectan

TOPOLOGÍA

Todas las VMs se conectan a una misma **red interna** de VirtualBox llamada `intnet` (una red privada, sin salida a Internet). El controlador `ryu` gobierna al switch `ovs`, y por ese switch pasa el tráfico de los tres hosts.

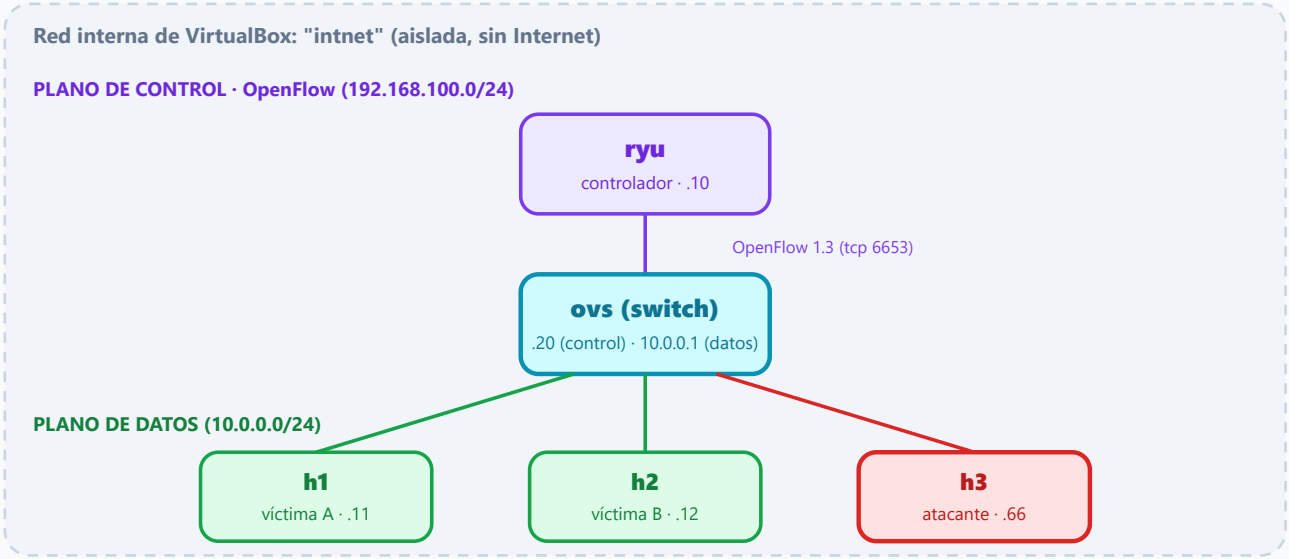


Figura 2 — Topología del laboratorio: un controlador, un switch y tres hosts sobre la red interna `intnet`.

¿Por qué SDN? En una red definida por software (SDN) el "cerebro" (el controlador) está separado del switch. Eso nos permite, en la fase de defensa, programar al controlador para que **detecte el ataque y lo bloquee automáticamente** — justo lo que hace un equipo de seguridad moderno.

0 Lo que necesitas

REQUISITOS

Equipos (hands-on en grupos)

- 60 estudiantes → grupos de **3-4** → **15-20 portátiles**.
- Cada portátil corre las **5 VMs** a la vez.
- **16 GB de RAM** recomendado, 4+ núcleos, ~40 GB libres.
- **Virtualización activada** en la BIOS/UEFI (VT-x / AMD-V).
- +1 equipo para el instructor (proyección).

Software (todo libre)

Programa	Versión
VirtualBox (host)	7.0.x / 7.1.x
Ubuntu Server (VMs)	22.04 LTS
Python (Ryu)	3.9
Ryu / eventlet	4.34 / 0.30.2
Open vSwitch	2.17.x
Scapy / netfilterqueue	≥2.5 / ≥1.0

GNS3 ya NO se usa. La versión anterior dependía de GNS3 y daba problemas de conectividad. Ahora todo se resuelve con VirtualBox + los scripts del repositorio. No lo instales.

Duración estimada ≈ 2 horas (con las VMs ya preparadas y repartidas): 30 min de charla · 15 min de arranque · 40 min de ataque (ver + alterar) · 20 min de defensa · 15 min de preguntas.

1 Preparación (una sola vez)

INSTRUCTOR

Esto lo hace el instructor **antes** de la charla. La idea: instalar todo en **una** VM base, clonarla 5 veces y configurar las redes de forma automática.

1. **Crea la VM base.** Instala **Ubuntu Server 22.04** en una VM nueva (instalación mínima, marca OpenSSH).

2. **Descarga el laboratorio e instala todo** dentro de la VM base. Un solo comando instala Ryu, Open vSwitch, Scapy y lo demás:

```
git clone https://github.com/juanortizpa/demostracion-narino.git
cd demostracion-narino
bash lab.sh instalar   # tarda unos minutos
```

   No hace falta dar permisos con `chmod` : el script lo hace solo.

3. **Apaga y clona la base 5 veces** en VirtualBox (clic derecho → Clonar). Marca "*Generar nuevas direcciones MAC*". Nómbralas: `ryu` , `ovs` , `h1` , `h2` , `h3` .

4. **Configura las redes automáticamente** desde Windows (PowerShell). Esto pone las 5 VMs en la red interna `intnet` y le da 2 adaptadores a `ovs` :

```
cd demostracion-narino\host-windows
.\crear-vms.ps1
```

   Opcional: `.\crear-vms.ps1 -Base "nombre-de-tu-base"` clona también desde PowerShell.

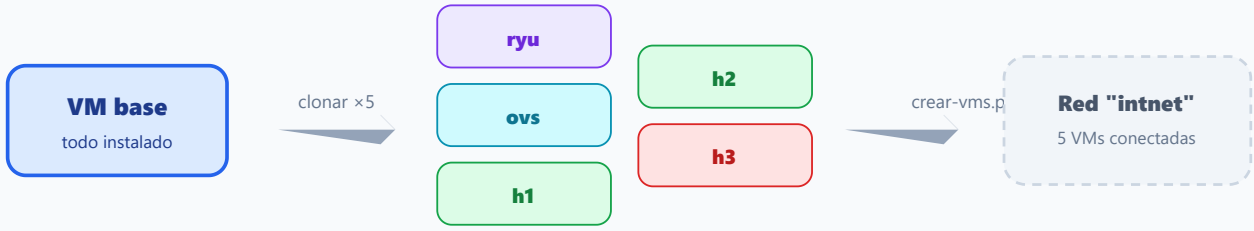


Figura 3 — Instalar una vez → clonar 5 → conectar la red automáticamente.

> **Para no instalar en clase:** exporta la VM base (o las 5) a un archivo **OVA** (*Archivo → Exportar servicio virtualizado*) y repártelo por USB a cada grupo. Así solo tienen que importar y encender.

2 En clase: arrancar el lab

CADA GRUPO

Cada grupo enciende las 5 VMs. En **cada VM** hay un único comando. La primera vez pide el rol, asigna la IP sola y la deja fija.

```
cd demostracion-narino
bash lab.sh           # elige: 1=ryu  2=ovs  3=h1  4=h2  5=h3
```

Orden de arranque (importante)

1. En `ryu` → opción **1** (controlador normal). Déjalo corriendo.
2. En `ovs` → opción **1** (conectar el switch al controlador).
3. En `h1` `h2` `h3` → ya tienen IP; listos para la demo.

Comprobación rápida: desde `h1`, el menú tiene la opción de "generar tráfico de prueba". Si `h2` responde al ping, la red está lista.

Si una VM pierde su IP: ya no debería pasar (queda fija con netplan). Si ocurriera, corre `bash lab.sh ip`. Para un diagnóstico completo: `bash scripts/diagnose.sh`.

3 La demostración

EL MOMENTO BONITO

Aquí se cuenta la historia en vivo. Antes del ataque, el mensaje viaja directo de h1 a h2. Con el ataque, pasa por el atacante, que lo ve y lo altera.



Figura 4a — Sin ataque.

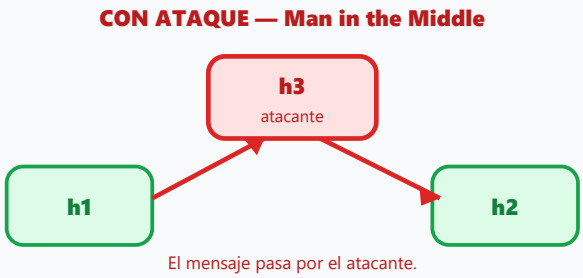


Figura 4b — El tráfico se desvía por h3.

Secuencia en vivo

1. **Abre el buzón en** `h2` → opción **2** (recibir mensajes). Déjalo abierto.
2. **Envía desde** `h1` → opción **1**, escribe un mensaje. Aparece en h2. (comunicación normal)
3. **Ataca y VE, desde** `h3` → opción **1**. El ARP spoofing se activa solo. Repite el envío: el atacante **ve** el mensaje pasar.
4. **Ataca y ALTERA, desde** `h3` → opción **2**. Escribe la palabra a buscar y su reemplazo. Repite el envío: a h2 le llega el mensaje **modificado**.
5. **¡La defensa! En** `ryu` : detén el controlador (Ctrl+C) y elige opción **2** (mitigación). Repite el ataque desde h3: Ryu **lo detecta y lo bloquea**.

> **En el atacante ya no hacen falta dos terminales:** el ARP spoofing se enciende y se apaga solo con cada opción. Para cortar el ataque a mano, usa la opción **3**. Al terminar, la red se restaura.

✓ Problemas comunes y checklist

CIERRE

Síntoma	Solución
Las VMs no se ven entre sí	Confirma que las 5 están en la red interna `intnet` (lo hace `crear-vms.ps1`). Prueba `ping 10.0.0.12` desde h1.
`bad interpreter: ^M` al correr un script	Ya resuelto: el repo fuerza finales de línea LF. Si editaste el archivo en Windows, vuelve a clonarlo.
El switch no conecta con el controlador	Arranca primero **ryu** (opción 1), luego **ovs** (opción 1).
Perdí la IP al reiniciar	Corre `bash lab.sh ip`. (Normalmente queda fija sola.)
Diagnóstico general	`bash scripts/diagnose.sh`

Checklist antes de la charla

- [ ] VM base creada con Ubuntu 22.04
- [ ] `bash lab.sh instalar` ejecutado sin errores
- [ ] 5 clones creados (ryu, ovs, h1, h2, h3)
- [ ] `crear-vms.ps1` ejecutado
- [ ] Prueba completa: enviar → ver → alterar → bloquear
- [ ] OVA exportado y copiado a los USB
- [ ] Portátiles de los grupos con 16 GB y VT-x activado
- [ ] Proyector probado con el equipo del instructor

> **Recuerda el marco ético:** todo se hace en un entorno virtual y aislado, sin datos reales ni redes de terceros. El objetivo es **aprender a defender**.